# Roteiro 8 — Configuração, Docker e pipeline

**Disciplina:** Desenvolvimento Web com .NET — UNIP. **Fonte:** Roteiros.pdf, páginas 24–26.
**Consolidação:** 06/11/2026, com preservação das evidências de outubro e de 05/11.

## Comprovação remota e fechamento de E4.2 — 06/11/2026

**Roteiro 8: 29 requisitos concluídos e verificados e duas extensões alternativas não aplicáveis fundamentadas.** O CI 42313806838, tentativa 2 terminou aprovado para o head `90ad9a2f6241a520df38d3be9b77a60fc76df9db`, testado no merge sintético `731aadb3d531e22820f086f472f7ddad8b5151db`. Foram aprovados 55 testes .NET e 33 React, zero falhos e zero ignorados, além de SQL/migrations, contrato/cliente, builds, Docker com HTTPS e usuário não privilegiado, smoke, saúde nativa e save/upload. A etapa sem fallback consumiu o Actions Secret para gerar o PFX; o contêiner abriu esse certificado e respondeu por HTTPS, com manifesto nativo `healthy`. O agente não consultou o valor do segredo; ele não foi incluído nas evidências, imagem ou pacote. A comprovação sanitizada (../evidencias/e4-2-20261106/github/comprovacao-e4-2.json) identifica etapas, testes, head/checkout, artefato e os manifestos de smoke/saúde. A imagem e sua versão usam o checkout sintético; a árvore de fontes é igual à do head enviado.

A tentativa 1 foi reprovada pela guarda que exige ao menos 16 caracteres no Actions Secret, antes de executar o contêiner, smoke, saúde nativa e entrega. Os testes anteriores da execução haviam passado; essa rejeição foi de configuração. O responsável informou a correção do segredo e a tentativa 2 reexecutou o mesmo head/checkout. Os registros da primeira tentativa (../evidencias/e4-2-20261106/github/comprovacao-e4-2-attempt1.json) permanecem preservados; não são substituídos pelo resultado aprovado posterior.

O secret store escolhido é GitHub Actions Secrets, conforme a alternativa do PDF, página 25. A senha do certificado é obrigatória, sem fallback; o certificado é efêmero, montado externamente e usado no HTTPS real da imagem. A aprovação remota acrescenta a prova que faltava à verificação local com stubs. O segredo foi configurado pelo responsável, que autorizou commit/push; o agente não consultou seu valor, que não integra as evidências, imagem ou pacote. Não foi necessário criar provedor fictício, OIDC sem destino, GHCR ou implantação pública.

Na matriz completa há 233 concluídos e verificados, 13 não aplicáveis fundamentados e duas limitações históricas (G-02/G-03). O estado global é **ENTREGA PREPARADA COM PENDÊNCIAS HISTÓRICAS**, pois os marcos anteriores exigidos na página 2 não podem ser comprovados retroativamente. As 12 capturas reais e os oito relatórios permanecem identificados, preservando todas as evidências anteriores. O pacote novo é preparado em `artifacts/entrega-e4-2-20261106/`, com fontes versionadas limpas, inventário, inspeção, extração e validação, conforme entrega final (../ENTREGA_FINAL.md).

A aprovação remota da revisão efetivamente empacotada deve ser conferida nos checks do PR e no comprovante final associado ao SHA registrado no manifesto do pacote. A execução citada comprova seus próprios head e checkout; não aprova automaticamente commits posteriores. O comprovante do CI final e os dados de integridade do ZIP ficam nos arquivos de acompanhamento ao lado do pacote, sem

alterar novamente as fontes versionadas. O PR permanece sem merge. Integração e CI da `main` só são apresentados quando houver autorização específica e execução real. O ensaio negativo anterior mantém sua validade para a ordem de publicação preservada.

As seções seguintes preservam os resultados das rodadas anteriores, incluindo o estado antes do run que comprovou E4.2. Suas contagens e pendências são históricas e não substituem este fechamento.

## Histórico de 06/11 — fechamento antes do CI de E4.2

**Estado: entrega preparada com pendências.** A aplicação existente foi preservada. A regressão nova aprovou 55 testes .NET com SQL exclusivo e 33 React, zero falhos/ignorados; o restore/build final ficou com zero avisos e erros. Seed repetido, contrato HTTPS/cliente gerado, CSRF/login/criação/logout e smoke do mesmo artefato em Development/Production passaram. Evidências novas (../STATUS_DE_EXECUCAO.md) e 12 capturas reais (../PLANO_CAPTURAS_FINAIS.md) identificam versões, comandos e limites. Os PDFs 02/04/05 incorporam as novas imagens; os demais relatórios técnicos foram preservados.

E4.2 foi implementado com o secret store nativo GitHub Actions: `PORTAL_CI_CERTIFICATE_PASSWORD` é obrigatório para gerar e abrir o PFX HTTPS do contêiner de teste, sem fallback. Checkout não persiste credenciais e `GITHUB_TOKEN` conserva `contents: read`. Não houve criação de provedor fictício, PAT, GHCR ou hospedagem pública. A decisão técnica e procedimento (../E4_2_SEGREDOS.md) detalham as alternativas. A checagem local do bloco e a validação estática não comprovam consumo do cofre GitHub: configurar o secret na conta e executar o workflow enviado ainda falta. R8-17 está implementado sem execução remota.

O CI 42123968355 foi reconsultado e está aprovado para o head remoto `5bf8fefcdd2b7b02dfb7012f8663e584a530b79e`, testado no merge sintético `d59527f578882c83c87e58cc5fafa6aa42c7787b`. Docker/SQL/HTTPS, usuário não privilegiado, smoke/saúde nativa e upload passaram. O PR #1 (../REVISAO_PR_1.md) continua draft, aberto e sem conflitos; `main` segue em `1da2eea`. Novas alterações locais ainda não têm CI remoto. Não houve push ou merge; o ensaio negativo anterior continua válido, sem alterar a ordem de publicação.

Os 248 requisitos foram conferidos: 232 concluídos/verificados, 13 não aplicáveis fundamentados, duas limitações históricas e um implementado sem execução remota. O pacote acadêmico inclui fontes, testes, migrations, contratos, scripts, oito PDFs e evidências, conforme entrega final (../ENTREGA_FINAL.md); inventário SHA256, inspeção de segurança, extração e restore/build da cópia são verificações separadas da compactação. O estado global permanece parcial por E4.2/CI próprio e limitações históricas, sem declarar 100% ou pronto para entrega.

As seções seguintes preservam os resultados e as decisões da rodada de 05/11 e de outubro. Afirmações históricas de ausência de capturas/ZIP referem-se àquelas rodadas; não substituem este fechamento adicional.

## Objetivo, introdução e ferramentas

Separar código/configuração, empacotar API e frontend e impedir publicação de uma versão que falhe nas verificações. A implementação utiliza arquivos por ambiente, User Secrets/variáveis externas, Docker multi-stage, Compose, GitHub Actions e scripts PowerShell.

## Configuração e ambientes

`appsettings.json` contém valores comuns e arquivos Development/Production contêm diferenças. Segredos ficam externos. `PortalOptions` valida o modo de armazenamento e a origem; a conexão é obrigatória para SQL Server. O modo Memory é restrito a Development/Testing e não representa uma alternativa de produção ao banco exigido.

O README lista variáveis, configuração local, seed e inicialização. A mesma fonte deve iniciar em desenvolvimento e produção simulada por configuração, sem alterar código. Esse cenário exige execução própria e não é comprovado apenas pela presença dos dois arquivos JSON.

## Containerização e escolha de implantação

O Dockerfile usa estágios para frontend, backend e runtime, imagens base versionadas e usuário não privilegiado no processo final. O React é incorporado em `wwwroot/app`. O Compose fornece SQL Server, Web, portas locais, certificado externo, volume SQL e persistência das chaves Data Protection.

| Estratégia | Consequência para o portal |
| --- | --- |
| React integrado à API | Um artefato operacional, mesma origem, cookies e roteamento mais simples |
| React em implantação separada | Releases independentes, exigindo configuração adicional de proxy/origem/sessão |

A estratégia integrada foi selecionada pelo escopo acadêmico e pela presença simultânea de MVC/SPA. Nenhuma credencial deve ser copiada para a imagem; o PFX e o ambiente são montados externamente. A evidência obrigatória inclui build, execução do contêiner e Health Check aprovado.

## CI, dependências e bloqueio de publicação

O workflow declara gatilhos de PR/branch principal e SDK/Node explícitos. A validação deve provisionar SQL Server dedicado, aguardar readiness, aplicar migrations no banco reservado e executar a suíte de integração. `Test-Integration.ps1` recusa banco fora do prefixo de teste e exige TRX com suítes obrigatórias, sem falhas/ignorados.

Frontend executa `npm ci`, auditoria de vulnerabilidades, tipos/contrato, testes e build. O cliente gerado precisa corresponder a OpenAPI real. Empacotamento/publicação ficam depois das verificações; uma falha deve encerrar o fluxo. O caminho aprovado já foi executado. A prova negativa foi executada em branch isolada e confirmou que nenhum artefato publicável foi produzido para o commit reprovado; um upload de diagnóstico de falha não equivale à publicação da aplicação.

A validação estática histórica (../evidencias/finalizacao-r8-20261029/ci-validacao-estatica.json) conferiu parser YAML, sintaxe Bash/PowerShell, SQL pronto antes das migrations/testes e smoke antes de salvar imagem/publicar artefatos, sem `always` ou `continue-on-error` na publicação. Esse método isolado não inicia Docker nem GitHub Actions. A evidência remota abaixo acrescenta a execução real; a validação do workflow alterado nesta auditoria continua distinta do run anterior.

O ensaio negativo (../ENSAIO_FALHA_CI.md) foi autorizado e executado com falha intencional em teste simples, exclusivamente em branch experimental. O run 42122062762 comprovou o bloqueio: publicação em pasta, imagem, smoke, save e upload de entrega ficaram `skipped`, e nenhum artefato publicável foi criado. A falha foi removida pelo commit de reversão enviado após registrar o resultado negativo; a recuperação 42122508347 voltou a `success`, com as suítes normais e a publicação aprovadas. A `main` e a branch de auditoria não receberam o teste defeituoso.

O aprimoramento selecionado é o inventário reproduzível de dependências .NET/npm como artefato. Tag da imagem, versão informativa e manifesto de commit relacionam artefato ao código validado.

Os inventários .NET (../evidencias/dependencias-dotnet.json) e npm (../evidencias/dependencias-npm.json) locais foram preservados; a etapa equivalente de inventário do runner também foi aprovada. A relação da imagem construída ao commit foi confirmada nos logs de build e nos metadados do artefato.

## Execução real aprovada no GitHub

O workflow 42117773881, `Validar e empacotar Portal de Tarefas`, foi consultado novamente pela API GitHub. Ele executou por `push` em `main`, commit **`1da2eead374d03fcfb62e5b0d7af17e2dbaf1cb2`**, com estado `completed` e conclusão `success`. O intervalo registrado foi 05/11/2026, 22:00:55–22:03:18 UTC, equivalente a 19:00:55–19:03:18 de São Paulo. O job `validar`, ID `127561862888`, aprovou todas as etapas obrigatórias; somente as duas etapas condicionais de diagnóstico de falha foram ignoradas por não haver falha.

| Verificação remota | Resultado confirmado |
|---|---|
| Restore/build .NET Release | Aprovado, zero avisos e zero erros |
| SQL Server, migrations e integração | Banco exclusivo; 55 aprovados, zero falhos e zero ignorados |
| Frontend | `npm ci`, auditoria, 33 testes aprovados em cinco arquivos, typecheck e build |
| Contrato | OpenAPI real, cliente gerado, comparação sem divergência e chamada TypeScript aprovados |
| Imagem | Build multi-stage integrado aprovado, tag com SHA completo |
| Execução | SQL Server e HTTPS; a etapa verificou que o UID do processo não era zero |
| Smoke | Liveness/readiness, `/version`, MVC, SPA e assets aprovados pelo script |
| Empacotamento | `docker save` e upload executados depois dos gates obrigatórios |

Os logs identificam a imagem `portaltarefas:1da2eead374d03fcfb62e5b0d7af17e2dbaf1cb2`, ID local Docker `sha256:c80953ffbbb3ff59232094808f251b33fd40375fc9d376e48130221944e1f80c`. A referência normalizada `docker.io/library/portaltarefas` na saída do build é o nome local do Docker e não comprova push ao Docker Hub ou GHCR.

O artefato 13456822922 chama-se `portaltarefas-1da2eead374d03fcfb62e5b0d7af17e2dbaf1cb2`, tem **159.303.801 bytes** e digest do ZIP **`sha256:650797a7b10c8b54271d8d4885626f40b091ef59c251a47a404007643e711e2e`**. A API informa expiração em 12/11/2026, 22:02:52 UTC. Essa retenção curta exige preservar os registros acadêmicos; a existência posterior do download não é garantida.

Foram preservados resumo e limites (../evidencias/auditoria-final-20261105/github/execucao-aprovada.json), etapas (../evidencias/auditoria-final-20261105/github/jobs-e-etapas.json), metadados do artefato (../evidencias/auditoria-final-20261105/github/artefatos.json) e trechos de resultados do log (../evidencias/auditoria-final-20261105/github/trechos-log.json). O ZIP completo não foi extraído nesta consulta; os valores individuais de UID e do manifesto smoke não foram lidos de seu interior. A aprovação das etapas e o script executado confirmam as verificações e seus critérios de sucesso.

O smoke exige respostas HTTP 200 de live/ready e versão exata. O run aprovado não registrou explicitamente o estado nativo `.State.Health.Status` do Docker. Esta auditoria acrescentou ao CI a espera de `healthy` antes de salvar/publicar a imagem; o resultado remoto foi aprovado no CI do PR descrito adiante, separadamente da execução histórica.

## Bloqueio após falha comprovado remotamente

O run 42122062762 executou por push autorizado em `experimento/ci-bloqueio-20261105`, commit **`1423c195e52ad2aaa75fdcfab562ba53456899dc`**, entre 22:30:28 e 22:31:57 UTC de 05/11/2026 (19:30:28–19:31:57 de São Paulo). A conclusão `failure` é a esperada neste cenário: a etapa **Aplicar migrations e executar testes sem ignorados** reportou **55 aprovados, uma reprovação sintética e zero ignorados**. O nome do teste foi `CiPublicationGateTests.FalhaControladaDeveInterromperPublicacao`; a infraestrutura anterior estava aprovada.

As seis etapas de publicação em pasta, build da imagem, execução da imagem, smoke, save e upload de entrega ficaram `skipped`. O frontend não foi executado após a interrupção. A API de artefatos informou somente `diagnostico-1423c195e52ad2aaa75fdcfab562ba53456899dc`, ID `13458922164`, 66.567 bytes; o artefato publicável esperado estava ausente. Esse diagnóstico permite examinar a reprovação sem publicar a aplicação. O resumo (../evidencias/auditoria-final-20261105/github/ensaio-negativo-execucao.json), as etapas (../evidencias/auditoria-final-20261105/github/ensaio-negativo-jobs.json), os artefatos (../evidencias/auditoria-final-20261105/github/ensaio-negativo-artefatos.json) e os resultados do log (../evidencias/auditoria-final-20261105/github/ensaio-negativo-trechos-log.json) comprovam **V3**.

A reversão `943e944a13d39c08bc82b9bb314c13986f5674b3` foi enviada após a execução negativa terminar, sem force push. A recuperação 42122508347 terminou `completed/success` em 05/11/2026, 22:33:37–22:36:36 UTC (19:33:37–19:36:36 de São Paulo), com **55 testes .NET e 33 React aprovados, zero falhos e zero ignorados**, build/contrato, imagem SQL/HTTPS, smoke e upload aprovados. O artefato 13460037445, `portaltarefas-943e944a13d39c08bc82b9bb314c13986f5674b3`, tem 159.295.700 bytes, digest `sha256:bd8d384b0d5b0f7caaf386c4ce3206714e23db586c093af8e32429adeeb286ff` e expiração em

12/11/2026 às 22:36:06 UTC. O resumo de recuperação (../evidencias/auditoria-final-20261105/github/ensaio-recuperacao-execucao.json), as etapas (../evidencias/auditoria-final-20261105/github/ensaio-recuperacao-jobs.json), os artefatos (../evidencias/auditoria-final-20261105/github/ensaio-recuperacao-artefatos.json) e os trechos de log (../evidencias/auditoria-final-20261105/github/ensaio-recuperacao-trechos-log.json) comprovam o retorno da suíte normal sem o teste sintético. O draft PR 1 teve execução própria aprovada para o novo gate nativo, descrita abaixo. O ensaio experimental utiliza a base anteriormente aprovada e não comprova esse novo gate. A `main` continua preservada, sem merge experimental.

## Novo gate de saúde aprovado no CI do PR

O run 42122467632, evento `pull_request`, terminou `completed/success` em 05/11/2026, 22:33:20–22:36:40 UTC (19:33:20–19:36:40 de São Paulo). Na execução analisada do draft PR 1, o head da branch era **`bfc46abbf99737724055af555bd785939bb795b8`**, com base `1da2eead374d03fcfb62e5b0d7af17e2dbaf1cb2`. O GitHub executou o **merge sintético `3f4c18a0c47ba70d3a8190f4cc0437257d5d987b`**, que corresponde a `github.sha` e identifica a imagem e o artefato deste run. O head do PR e o SHA usado no artefato têm funções diferentes; o PR continua draft, sem merge na main.

Foram aprovados **55 testes .NET e 33 React, zero falhos/ignorados**, restore/build, contrato/cliente, typecheck/build, imagem Docker, SQL/HTTPS e smoke. A etapa 27, **Confirmar Health Check nativo do contêiner**, terminou `success`; a etapa 28 salvou a imagem e a etapa 29 publicou o artefato depois desse gate. A conclusão de `healthy` é uma **inferência declarada**: o script da etapa somente retorna código zero quando `docker inspect .State.Health.Status` responde healthy, e rejeita unhealthy ou timeout. O valor não foi impresso no log, e o manifesto `container-health.json` dentro do ZIP não foi extraído; os limites não são ocultados.

A imagem foi identificada por `portaltarefas:3f4c18a0c47ba70d3a8190f4cc0437257d5d987b`, ID local `sha256:9661b337586cbc643067fe9c6da8988000ab0bb4da62ee0bb731232a6721266b`. O artefato 13458418239, `portaltarefas-3f4c18a0c47ba70d3a8190f4cc0437257d5d987b`, tem **159.298.190 bytes**, digest do ZIP **`sha256:c8d75257563beda8cc831f91e764cf9c107e156312897fd9aae8a118461a369f`** e expiração em 12/11/2026 às 22:36:11 UTC. Execução/SHAs (../evidencias/auditoria-final-20261105/github/ci-saude-nativa-execucao.json), etapas (../evidencias/auditoria-final-20261105/github/ci-saude-nativa-jobs.json), artefatos (../evidencias/auditoria-final-20261105/github/ci-saude-nativa-artefatos.json) e resultados/limites (../evidencias/auditoria-final-20261105/github/ci-saude-nativa-trechos-log.json) preservam a evidência sem versionar o ZIP ou logs brutos.

## CD, promoção e rollback

`Publish-Local.ps1` prepara homologação com imagem local identificada, arquivo externo de ambiente e certificado. O script preserva a instância anterior e chama `Smoke-Test.ps1`, que verifica live/ready, versão, MVC e entrada da SPA. O procedimento não autoriza publicação remota automática.

Para um provedor futuro, conexões/credenciais devem vir de secret store ou identidade federada; parâmetros do destino não são inventados. A promoção reutiliza a imagem já testada, mantendo commit/tag. O rollback deve voltar à imagem anterior preservada e considerar separadamente compatibilidade das migrations e backup do banco. Volumes não devem ser apagados para reverter uma aplicação.

Na página 25, E4 exige imagem vinculada ao commit, identidade federada **ou** secret store do provedor, implantação em ambiente de teste com smoke/Health Check e estratégia de promoção/retorno com registro de versão. O PDF não exige GHCR, Docker Hub ou um provedor específico: a distribuição pode usar o artefato de imagem já validado. A implantação em staging com aprovação antes de produção aparece na página 26 como extensão alternativa. O inventário de dependências é a extensão selecionada, de modo que não se acrescenta infraestrutura de produção apenas por existir no projeto de referência.

O ambiente efêmero do runner implantou a imagem com SQL Server e HTTPS, verificou saúde, executou smoke e identificou a versão pelo SHA. Essa execução atende **E4.3, implantação em ambiente de teste**; a homologação local atual do mesmo artefato reforça a verificação. O PDF não determina que o ambiente seja permanente ou remoto. Não houve hospedagem persistente, promoção para produção ou rollback remoto; esses cenários não são apresentados como executados nem transformados em exigências adicionais. **E4.2, identidade federada ou secret store do provedor, continua pendente**, pois a configuração externa e as senhas efêmeras do CI não demonstram esse mecanismo. A estratégia de promoção/retorno e os scripts estão implementados; a promoção reutiliza exatamente a imagem validada, sem reconstruí-la.

## Histórico de 05/11 — finalização sem novas capturas

Nesta rodada foram preservados os sete relatórios anteriores e todas as imagens existentes. O plano visual final (../PLANO_CAPTURAS_FINAIS.md) é apenas um checklist para execução posterior. Não foram instaladas ferramentas de captura nem executadas automações de navegador para screenshots nesta etapa.

O diagnóstico Docker Windows (../DOCKER_WINDOWS.md) e seu registro de comandos de 29/10 (../evidencias/finalizacao-r8-20261029/docker-ambiente.json) comprovam Docker CLI 29.9.2 e Compose 5.6.1 portáteis. A configuração Compose rejeitou variáveis obrigatórias ausentes e aceitou marcadores sintéticos definidos somente no processo. `docker info` retornou código 1 por ausência de Engine/WSL. A reprodução local continua dependendo desses pré-requisitos; o build, a execução, o usuário não privilegiado e o smoke no runner já foram comprovados pela execução de 05/11. Nenhuma instalação global foi feita para resolver o bloqueio Windows.

A configuração de homologação separa o projeto `portal-tarefas-tests` e exige banco reservado de teste. O namespace anterior pode ser mantido explicitamente por `PORTAL_COMPOSE_PROJECT=portal-tarefas`, preservando seus volumes. O script de publicação confere SHA completo da imagem contra a revisão OCI, valida o banco, aplica migrations apenas mediante opção explícita, compartilha chaves quando configurado e registra manifesto sanitizado e comandos de rollback. As guardas e sua execução negativa são verificações próprias; não substituem uma publicação Docker real.

## Resultados consolidados e limites

A regressão local de 05/11/2026 aprovou **55 testes .NET com SQL real e 33 testes React**, zero falhos, zero ignorados e zero bloqueados nessas duas suítes. Restore/build .NET Release, typecheck e build frontend passaram. O resumo SQL (../evidencias/auditoria-final-20261105/regressao/dotnet-

testes/20261105T221626427Z-9a7a84b4/summary.json) e os resultados frontend (../evidencias/auditoria-final-20261105/regressao/frontend-resultados.json) identificam os comandos e a execução. O banco exclusivo `PortalTarefas_Tests_Final20261105_26b194a7` não foi compartilhado com seed/homologação.

Em outro banco novo reservado, o seed repetido (../evidencias/auditoria-final-20261105/regressao/seed-idempotente.json) manteve três tarefas, três auditorias, duas roles, dois usuários sintéticos e três migrations. O contrato HTTPS (../evidencias/auditoria-final-20261105/regressao/contrato-real.json) e os tipos gerados permaneceram idênticos aos versionados. Cliente de leitura (../evidencias/auditoria-final-20261105/regressao/cliente-sql-https.txt) e cliente autenticado (../evidencias/auditoria-final-20261105/regressao/cliente-autenticado-https.txt) passaram com validação TLS. A publicação em pasta nos dois ambientes (../evidencias/auditoria-final-20261105/regressao/homologacao-local.json) retornou HTTP 200 em live/ready, MVC e SPA, versão exata do artefato de origem e documentação restrita em Production. Processos de teste próprios foram encerrados; dados e volumes preservados.

O ensaio local isolado (../evidencias/auditoria-final-20261105/ensaio-local/resumo.json) produziu uma reprovação intencional, sem ignorados, e a reversão restaurou 29 testes de domínio/memória aprovados. Esses 29 são um subconjunto da suíte normal, e a falha esperada não é uma regressão do produto. A branch experimental possui commits reais de preparação, falha e reversão descritos no procedimento. Sua execução negativa remota foi autorizada e comprovou o bloqueio; a confirmação da recuperação após a reversão já enviada é registrada separadamente.

A validação estática do CI atualizado (../evidencias/ci-validacao-estatica.json) aprovou 31 etapas YAML, 22 blocos Bash, seis scripts e quatro blocos PowerShell. O teste local do novo gate de saúde (../evidencias/auditoria-final-20261105/gate-health-local.json) aprovou três cenários com funções Docker/sleep simuladas: aceita `healthy`, rejeita `unhealthy` e timeout. Essa checagem da lógica não é evidência de Docker real. A consulta Docker no Windows (../evidencias/auditoria-final-20261105/docker-local.json) reconfirmou a indisponibilidade do Engine, preservando a aprovação remota anterior.

Os resultados e bloqueios estão consolidados no status de execução (../STATUS_DE_EXECUCAO.md). O CI remoto, o build/execução da imagem e a implantação efêmera de teste com smoke HTTP estão aprovados para o commit identificado. O novo gate de saúde nativo também foi aprovado no CI do PR. Permanecem a limitação de reprodução Docker no Windows sem Engine e E4.2, mecanismo de identidade/segredos do provedor ainda não demonstrado. A recuperação do ensaio já voltou a `success`. O bloqueio após falha controlada já foi comprovado remotamente.

O roteiro 8 permanece **parcial** por E4.2, identidade federada/secret store do provedor. V3, bloqueio de publicação após falha, já foi comprovado remotamente; a recuperação e o novo gate nativo estão aprovados em suas execuções próprias. Não foram feitas novas capturas; as imagens e os relatórios 1–7 permanecem preservados. O PDF 08 é gerado a partir deste Markdown pelo processo existente, com hash verificado e sem produzir screenshots.

Como histórico de 29/10/2026, os smokes Development (../evidencias/smoke-development.json) e Production simulada (../evidencias/smoke-production.json) foram executados sobre o mesmo binário publicado: live, ready, MVC e SPA retornaram 200 e `/version` informou o commit exato `47c5f2fe37768d16c1a009b9b15688941e04bb02`. Development recebeu User Secrets em perfil isolado e Production recebeu variáveis externas. O manifesto do artefato local histórico (../evidencias/artefato-local.json)

identifica DLL/assets e seus SHA256. Essa execução antiga comprova publicação local e configuração por ambiente; não é apresentada como execução Docker, workflow remoto ou CD por imagem. Os novos smokes de 05/11, vinculados ao SHA de origem `1da2eea`, estão identificados separadamente acima.